UNIVERSIDAD DE MONTERREY

Proyecto de Introducción a la Ingeniería

Actividad 2.4 Avances del Proyecto 5

Profesor Antonio Martínez Torteya

Nombre	Matricula	Código de Honor
Geu Davila Guerrero	656528	“Doy mi palabra que he realizado esta actividad con Integridad Académica” - Geu Davila Guerrero
Angel Gabriel Romero Patena	694550	“Doy mi palabra que he realizado esta actividad con Integridad Académica” - Angel Gabriel Romero Patena
Esaul Alejandro Villareal Abrego	646239	“Doy mi palabra que he realizado esta actividad con Integridad Académica” - Esaúl Alejandro Villarreal Ábrego

San Pedro Garza García, Nuevo León, México

11/10/2026

Introducción

El objetivo de esta actividad es explorar los fundamentos y el funcionamiento del sensor ultrasónico, de manera que se posible crear un sistema que permita la medición de distancias en un rango determinado. Este tipo de sensor puede servir para realizar diversas tareas de control y automatización como lo son la navegación y mapeo, los cuales usan este tipo de sensor para evitar obstáculos y trazar mapas que les permita explorar un entorno.

En este proyecto se elaboró un sistema de seguridad basado en el sensor ultrasónico. Con base en la distancia del objeto de medición, se determina la velocidad a la que suena un buzzer, replicando el funcionamiento de las cámaras de reversa en los automóviles modernos. Con base en esto último, el sistema también fue modificado para activarse sólamente cuando se ponga un sistema de control de motores (ver **Proyecto 3**) en reversa.

Marco Teórico

Sensor Ultrasónico HC-SR04

Este sensor utiliza 4 pines: GND, VCC, TRIG y ECHO. GND y VCC suministran energía eléctrica al sensor mientras que TRIG y ECHO controlan a este último. Al activar TRIG por 10 microsegundos, el sensor envía un pulso ultrasónico. Mediante la función pulseIn, que toma como referencia el valor del pin ECHO, permite medir el tiempo total entre que se mandó el pulso y se recibió. Conociendo la velocidad del sonido (340 m/s), se puede convertir este valor de tiempo en un valor de distancia que se puede usar en un programa (Naylamp Mechatronics, s.f.).

Metodología

Circuito

A continuación, en la **Figura 1**, se muestra un diagrama del circuito utilizado para el proyecto.

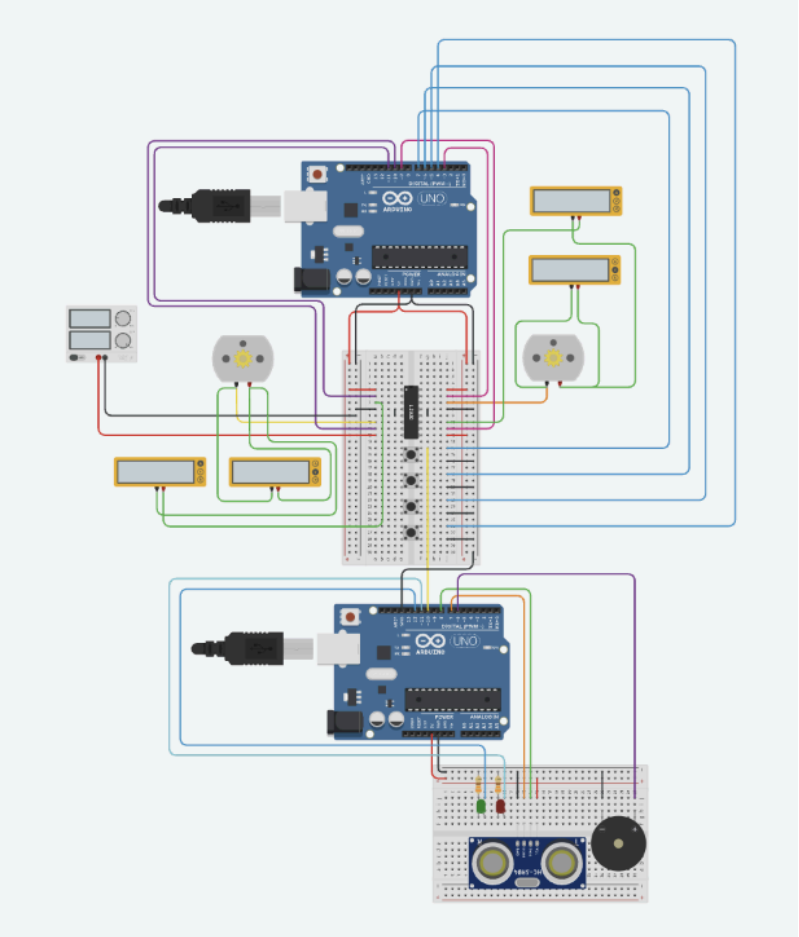


Figura 1: Diagrama del circuito utilizado para el proyecto (autoría propia).

Se hizo uso de un segundo Arduino dado a que uno solo no tiene la cantidad de pines necesarios para operar todos los sensores y actuadores. Para conectar el Arduino encargado del sensor ultrasónico y del buzzer (referido a partir de ahora solo como Arduino) al resto del sistema, se utilizó un cable que conecta el pin 10 al botón que corresponde a la reversa de los motores. Hubo una dificultad al momento de hacer que el Arduino detectara si se había presionado el botón dado a que no se había conectado a la misma tierra, lo cual es necesario para que puedan usar la misma referencia de tierra y comunicarse entre sí (Jain, 2026).

Con base en la señal recibida en el pin 10, el Arduino usa los pines 11 y 12 para controlar dos LEDs: uno verde que indica que el sistema está inactivo y otro rojo que indica lo contrario. Cuando se activa el sistema, el Arduino también utiliza los pines 7 y 8 para controlar las funciones de Trigger y Echo del sensor ultrasónico, así como el pin 6 para hacer sonar al buzzer.

Programación

El código utilizado se puede observar en la **Figura 2**. Tras definir y configurar los pines utilizados en el circuito, el programa entra en la fase loop, donde constantemente checa si el botón de reversa ha sido presionado. Se configura al pin 10 como INPUT PULLUP dado a que, cuando se presiona el botón, el pin queda conectado a tierra.



```
// C++ code
//

int activate = 10;
int b = 6;
int red = 11;
int green = 12;
int trig = 8;
int echo = 7;
int buzz_length;
long duration;
long distance;

long ultrasonic();
void buzz(int d);
void setup()
{
  pinMode(activate, INPUT_PULLUP);
  pinMode(b, OUTPUT);
  pinMode(red, OUTPUT);
  pinMode(green, OUTPUT);
  pinMode(trig, OUTPUT);
  pinMode(echo, INPUT);
  Serial.begin(9600);
}

void loop()
{
    digitalWrite(red, LOW);
    digitalWrite(green, HIGH);

  while(digitalRead(activate) == LOW){
    digitalWrite(green, LOW);
  digitalWrite(red, HIGH);
    distance = ultrasonic();
    buzz(distance);
  }
}


long ultrasonic(){
  digitalWrite(trig, LOW);
  delay(2);
  digitalWrite(trig, HIGH);
  delay(10);
  digitalWrite(trig, LOW);
  duration = pulseIn(echo, HIGH);
  distance = duration * (0.0343/2);
  Serial.println(distance);
  return distance;
}

void buzz(int d){
  buzz_length = map(d, 2, 330, 10, 1000);
  delay(buzz_length);
  tone(b, 2000);
  delay(100);
  noTone(b);
}
```

Figura 2: Código utilizado para el proyecto (Autoría propia).

Si no se ha presionado, el LED verde se mantiene encendido y el LED rojo apagado. Cuando se presiona el botón, el LED rojo se enciende y el LED verde se apaga. Además se comienza a usar la función *ultrasonic()* para leer la distancia al objeto desde el sensor (convirtiendo la medición del tiempo transcurrido entre pulso y recepción a distancia mediante una conversión de unidades). Esta medición es dada a otra función denominada *buzzer()*, el cual convierte un rango de distancia entre 2 y 330 cm (medido en TinkerCad) en un rango de 10 a 1000 ms entre cada pitido del buzzer.

Conclusiones

La realización del proyecto permitió obtener un mejor entendimiento del funcionamiento y aplicación del sensor ultrasónico en el control de sistemas. Entre los puntos de mejora identificados se encuentra:

- Programar una advertencia en caso de que la distancia que el sensor está intentando medir no quede por afuera de su rango.
- Evitar el uso de delays para mejorar el flujo del programa y permitir una lectura más constante de la distancia.

Referencias

Jain, J. (2026, 11 de marzo). *Common Ground in Electronics: Why It Matters in Your Circuit*. Zbotic. Recuperado el 12 de octubre del 2026 de: https://zbotic.in/common-ground-in-electronics-why-it-matters-in-your-circuit/?st_source=ai_overview

Naylamp Mechatronics. (s.f.). *Tutorial de Arduino y sensor ultrasónico HC-SR04*. Recuperado el 12 de octubre del 2026 de: https://naylampmechatronics.com/blog/10_tutorial-de-arduino-y-sensor-ultrasonico-hc-sr04.html